

Batch: A2 Roll No.: 16010324044

Experiment / assignment / tutorial No. 3

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

TITLE: Process scheduling algorithms

Objective: Shell scripts programs on Process scheduling algorithms (preemptive type, Non preemptive)

OUTCOME: Student can Compare different algorithms used for management and scheduling of processes.

Implementation of Basic Process management algorithms -

- 1) Write full code for all FCFS, Shortest job first (non preemptive) algorithm along with output and screen shot.

FCFS Code:

```
#include <stdio.h>

int main() {

    int n = 4;

    int at[] = {0,1,3,5};

    int bt[] = {7,6,4,2};

    int ct[4], tat[4], wt[4];

    ct[0] = at[0] + bt[0];

    for(int i=1;i<n;i++){
```

Department of Electronics and Telecommunication Engineering

```
    if(ct[i-1] < at[i])

        ct[i] = at[i] + bt[i];

    else

        ct[i] = ct[i-1] + bt[i];

}

for(int i=0;i<n;i++){

    tat[i] = ct[i] - at[i];

    wt[i] = tat[i] - bt[i];

}

float avgWT=0, avgTAT=0;

printf("Process\tAT\tBT\tCT\tTAT\tWT\n");

for(int i=0;i<n;i++){

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",

           i+1,at[i],bt[i],ct[i],tat[i],wt[i]);

    avgWT += wt[i];

    avgTAT += tat[i];

}
```

```
printf("\nAverage Waiting Time = %.2f",avgWT/n);

printf("\nAverage Turnaround Time = %.2f\n",avgTAT/n);

return 0;
}
```

FCFS Output:

Process	AT	BT	CT	TAT	WT
P1	0	7	7	0	
P2	1	6	13	6	
P3	3	4	17	10	
P4	5	2	19	12	

Average Waiting Time = 7.00
Average Turnaround Time = 11.75

=== Code Execution Successful ===

SJF Code:

```
#include <stdio.h>

struct Process{

    int id, at, bt, ct, tat, wt, done;

};
```

```
int main(){

    struct Process p[4]={

        {1,0,7,0,0,0,0},

        {2,1,6,0,0,0,0},

        {3,3,4,0,0,0,0},

        {4,5,2,0,0,0,0}

    };

    int completed=0,time=0;

    while(completed<4){

        int idx=-1;

        for(int i=0;i<4;i++){

            if(p[i].at<=time && p[i].done==0){

                if(idx==-1 || p[i].bt<p[idx].bt)

                    idx=i;

            }

        }

    }
```

```
if (idx == -1) {  
  
    time++;  
  
}  
  
else {  
  
    time += p[idx].bt;  
  
    p[idx].ct = time;  
  
    p[idx].tat = p[idx].ct - p[idx].at;  
  
    p[idx].wt = p[idx].tat - p[idx].bt;  
  
    p[idx].done = 1;  
  
    completed++;  
  
}  
  
}  
  
float avgWT = 0, avgTAT = 0;  
  
printf("Process\tAT\tBT\tCT\tTAT\tWT\n");  
  
for (int i = 0; i < 4; i++) {  
  
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",  
  
        p[i].id, p[i].at, p[i].bt,  
  
        p[i].ct, p[i].tat, p[i].wt);  
  
    avgWT += p[i].wt;  

```

```
avgTAT+=p[i].tat;

}

printf("\nAverage Waiting Time = %.2f",avgWT/4);

printf("\nAverage Turnaround Time = %.2f\n",avgTAT/4);

return 0;

}
```

SJF Output:

```
Process AT  BT  CT  TAT WT
P1  0   7   7   7   0
P2  1   6  19  18  12
P3  3   4  13  10   6
P4  5   2   9   4   2

Average Waiting Time = 5.00
Average Turnaround Time = 9.75

=== Code Execution Successful ===
```

- 2) Consider the following snapshot of the process to be executed. Draw the Gantt chart and determine the average **waiting time** and average **turnaround time** for FCFS, SJF scheduling algorithm.

Process	Arrival Time	Burst Time
P1	0	7

K. J. Somaiya School of Engineering, Mumbai-77

P2	1	6
P3	3	4
P4	5	2

FCFS:

Gantt Chart

0 7 13 17 19

| P1 | P2 | P3 | P4 |

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	6	13	12	6
P3	3	4	17	14	10
P4	5	2	19	14	12

Average Waiting Time = 7.00

Average Turnaround Time = 11.75

SJF:

Gantt Chart

0 7 9 13 19

| P1 | P4 | P3 | P2 |

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	6	19	18	12

K. J. Somaiya School of Engineering, Mumbai-77

P3	3	4	13	10	6
----	---	---	----	----	---

P4	5	2	9	4	2
----	---	---	---	---	---

Average Waiting Time = 5.00

Average Turnaround Time = 9.75

Conclusion

In this experiment, I learnt about FCFS schedules processes strictly according to their arrival time, which is simple but can result in longer waiting times for shorter jobs. SJF (Non-Preemptive) selects the process with the smallest burst time among the available processes, reducing the average waiting and turnaround times.

Signature of faculty in-charge